

FinanceControl

Documentação Técnica — v2.6.0 · 15/05/2026

1. Visão Geral

Sistema de gestão financeira com classificação IFRS, para controle empresarial completo de receitas, despesas, cartões, folha de pagamento e relatórios.

Funcionalidades principais:

- Lançamentos de receitas e despesas com classificação IFRS
- Gestão de cartões de crédito (limite, fechamento, vencimento)
- Importação de faturas PDF de múltiplos bancos brasileiros
- Relatórios: DRE, fluxo de caixa, conciliação por categoria
- Gestão de funcionários e folha de pagamento (INSS, FGTS, bônus)
- Controle de categorias com agrupamento IFRS personalizável
- Múltiplos usuários com controle granular de permissões

Versão	URL	Repositório GitHub	Branch
Rotas Café (Prod)	financecontrol-kiyl.onrender.com	JuniorPaviani/rotas-cafe	main
Rotas Café (Staging)	rotas-cafe-staging.onrender.com	JuniorPaviani/rotas-cafe	dev
FinanceControl (Prod)	financecontrol-public.onrender.com	JuniorPaviani/financecontrol	main

2. Arquitetura

Monolito com SPA React servida pelo próprio FastAPI em produção. Frontend compilado durante o build e embutido como arquivos estáticos.

```
[ React 18 + Vite ] > REST/JSON > [ FastAPI 0.115 (Python 3.12) ]
|
SQLAlchemy ORM
|
[ PostgreSQL (Render / Neon.tech) ]
```

- Produção: frontend embutido no backend, uma única instância Render
- Desenvolvimento: Vite :5173 + Uvicorn :8000, CORS liberado para localhost
- Qualquer rota não-/api/ devolve index.html (SPA client-side routing)

3. Stack Tecnológica

Backend (Python 3.12.10)

Biblioteca	Versão	Finalidade
FastAPI	0.115.0	Framework web ASGI

Uvicorn	0.30.6	Servidor ASGI
SQLAlchemy	2.0.35	ORM
Pydantic	2.9.2	Validação / serialização
python-jose	3.3.0	JWT tokens
passlib + bcrypt	4.1.3	Hash de senhas
pdfplumber	0.11.4	Extração de texto de PDF
openpyxl	3.1.5	Exportação Excel
reportlab	4.5.1	Geração de PDF
psycopg2-binary	2.9.9	Driver PostgreSQL
Alembic	1.13.3	Migrações de banco de dados

Frontend (Node 20.18.0)

- React 18 + Vite
- Lucide React — ícones
- Recharts — gráficos
- XLSX / ExcelJS — exportação de planilhas

4. Estrutura de Diretórios

```

financecontrol/
├── backend/app/
│   ├── main.py # Entrypoint, lifespan, SPA serving
│   ├── models.py # Todos os modelos ORM
│   ├── schemas.py # Schemas Pydantic
│   ├── auth.py # JWT, hash, get_current_user
│   └── routers/ # auth, transactions, categories, cards,
│       └── # invoices, reports, employees, users
│   └── parsers/ # base, itau, bradesco, santander, caixa, banks
├── frontend/src/
│   ├── App.jsx # Roteamento de abas + guarda de autenticação
│   ├── hooks/useAuth.js # Token JWT, login/logout, persist localStorage
│   ├── api/client.js # apiFetch – wrapper HTTP com Bearer token
│   ├── styles/theme.js # Paleta de cores e helpers de estilo
│   ├── components/ # auth, dashboard, transactions, import, fatura,
│   │   └── # cards, categories, reports, employees, users
│   └── render.yaml # Blueprint Render (DB + serviço web)
├── render-build.sh # Build: npm install + npm run build + pip install
└── docker-compose.yml # Desenvolvimento local

```

5. Modelos de Dados

Modelo	Campos-chave	Relacionamentos
--------	--------------	-----------------

User	id, name, email, hashed_password role (admin user) can_view_reports, can_view_receitas force_password_change, is_active	→ transactions, cards categories, employees salary_payments
Transaction	id, user_id, category_id, card_id type (D/R), date, description, amount periodo_referencia, installment_* source (manual pdf_import)	→ User → Category → Card
Category	id, user_id, name, ifrs_group icon, color	→ User → transactions
Card	id, user_id, name, bank, last_four credit_limit, closing_day, due_day	→ User → transactions
Employee	id, user_id, name, cpf, position department, base_salary, status	→ SalaryPayment
SalaryPayment	base_salary, bonus, deductions inss, fgts, net_salary, status	→ Employee, User
ImportLog	batch_id, bank, filename total_rows, imported_rows, status	→ User
PasswordResetToken	token (64 chars), expires_at, used	→ User

6. API — Endpoints

Autenticação (/api/auth)

Método	Rota	Descrição
POST	/register	Cadastro + seed categorias padrão
POST	/login	Login (e-mail ou login alfanumérico) + senha → JWT
GET	/me	Retorna usuário autenticado
POST	/forgot-password	Envia e-mail com link de reset
POST	/reset-password	Redefine senha via token de e-mail
POST	/change-password	Troca senha + limpa force_password_change

Lançamentos (/api/transactions)

Método	Rota	Descrição
GET	/	Lista com filtros: período, tipo, cartão, busca
POST	/	Cria lançamento (suporte a parcelamento automático)
PUT	/[{id}]	Atualiza lançamento
DELETE	/[{id}]	Remove (ou grupo de parcelas inteiro)

Faturas PDF (/api/invoices)

Método	Rota	Descrição
POST	/upload	Upload PDF → auto-detecta banco → extrai transações

POST	/confirm	Confirma e persiste as transações extraídas
------	----------	---

Relatórios (/api/reports)

Método	Rota	Descrição
GET	/summary	Resumo mensal (N meses)
GET	/dre	DRE — Demonstração do Resultado
GET	/reconciliation	Conciliação por categoria
GET	/cashflow	Fluxo de caixa
GET	/export/excel	Exportação para Excel (.xlsx)

Funcionários (/api/employees)

Método	Rota	Descrição
GET	/	Lista funcionários
POST	/	Cadastra funcionário
PUT	/{{id}}	Atualiza dados
DELETE	/{{id}}	Remove funcionário
GET	/{{id}}/payments	Histórico de pagamentos
POST	/{{id}}/payments	Registra pagamento de salário

Usuários (admin) (/api/users)

Método	Rota	Descrição
GET	/	Lista usuários da conta
POST	/	Cria usuário com permissões
PUT	/{{id}}	Atualiza permissões / dados
DELETE	/{{id}}	Remove usuário

7. Autenticação e Segurança

- Tokens JWT (HS256) com expiração configurável (TOKEN_EXPIRE_HOURS, padrão 48h)
- Senhas com hash bcrypt via passlib
- SECRET_KEY gerado automaticamente pelo Render — nunca exposto no código
- Em ENVIRONMENT=production: Swagger UI e ReDoc desabilitados
- Reset de senha via token único de 64 chars com TTL de 1 hora
- force_password_change: admin criado via seed é forçado a trocar senha no 1° login

Roles de usuário:

- **admin** — acesso total, incluindo gestão de outros usuários
- **user** — acesso limitado pelas flags can_view_reports e can_view_receitas

8. Importação de Faturas PDF

Bancos com parser implementado: **Itaú** · **Bradesco** · **Santander** · **Caixa Econômica Federal** · **Sam's Club** · **Sicredi** · **Stone** · **Riachuelo (Midway)**

Fluxo de importação:

- 1. Upload do arquivo PDF pelo usuário na aba Importar
- 2. pdfplumber extrai o texto bruto do PDF
- 3. Auto-detecção do banco por regex nos parsers (método detect())
- 4. Parser específico extrai: data, descrição, valor, parcela (ex.: 02/12)
- 5. Frontend exibe prévia das transações com sugestão de categoria
- 6. Usuário revisa, ajusta categorias e confirma
- 7. Transações salvas com source='pdf_import' e batch_id único (UUID)
- 8. ImportLog registra: banco, arquivo, total_rows, imported_rows, status

9. Ambientes

Ambiente	Branch	ENVIRONMENT	Swagger	Log Level	URL
Produção (Rotas Café)	main	production	OFF	WARNING	financecontrol-kiyl.onrender.com
Staging (Rotas Café)	dev	staging	ON	INFO	rotas-cafe-staging.onrender.com
Produção (FinanceControl)	main	production	OFF	WARNING	financecontrol-public.onrender.com
Desenvolvimento local	—	development	ON	INFO	localhost:5173 / :8000

10. Deploy — Render.com

Comandos

```
Build: chmod +x render-build.sh && ./render-build.sh
```

```
Start: cd backend && uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

Variáveis de Ambiente

Variável	Obrigatório	Descrição
DATABASE_URL	Sim (prod)	Connection string PostgreSQL
SECRET_KEY	Sim	Chave JWT (gerada pelo Render automaticamente)
TOKEN_EXPIRE_HOURS	Não	Expiração JWT em horas (padrão: 48)
ENVIRONMENT	Não	"production" "staging" "development"
PYTHON_VERSION	Sim	3.12.10
NODE_VERSION	Sim	20.18.0
ADMIN_LOGIN	Não	Login do usuário admin master (seed no startup)
ADMIN_PASSWORD	Não	Senha provisória do admin (force_password_change=True)
FRONTEND_URL	Não	URL base para links de e-mail de reset de senha

11. Como Rodar Localmente

Opção A — Docker (recomendado)

```
Windows: duplo clique em INICIAR.bat
Linux/Mac: docker-compose up -d
Acesso: http://localhost:5173 (frontend)
http://localhost:8000 (backend / Swagger: /docs)
```

Opção B — Manual

```
cd backend
python -m venv .env && .env\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload

cd frontend
npm install && npm run dev
```

Arquivo backend/.env para desenvolvimento

```
DATABASE_URL=sqlite:///./financecontrol.db
SECRET_KEY=qualquer-chave-local-aqui
ENVIRONMENT=development
```